

Development and Verification of a Discrete Element Method Solver for Particle Dynamics

Pranjal Vats

SMME / Indian Institute of Technology , Mandi

Abstract—This paper reports an end-to-end discrete element method (DEM) workflow implemented in Fortran for spherical particles with gravity, particle–particle contact, and wall interaction inside a rectangular domain. The solver uses a linear spring-dashpot contact model with semi-implicit Euler time integration and is supported by Python post-processing for verification, profiling analysis, scaling plots, and report-ready figures. Verification covers free fall, constant-velocity transport, and damped bouncing behavior. Profiling identifies particle–contact evaluation as the dominant hotspot, motivating OpenMP parallelization with thread-safe force accumulation and validated serial-parallel equivalence. The final study includes measured speedup/efficiency trends, strong and weak scaling, a cell-linked neighbour-search comparison against all-pairs contact search, and timestep-sensitivity analysis with observed convergence behavior.

Index Terms—discrete element method, DEM, Fortran, particle simulation, verification, VTK, OpenMP

I. INTRODUCTION

The discrete element method is widely used to model granular systems in which particles interact through short-range contact forces. This project develops a complete DEM simulation and analysis pipeline, from governing equations and implementation through verification, profiling, shared-memory parallelization, and performance characterization.

The solver advances spherical particles in a rectangular box domain while neglecting rotational dynamics, which keeps the study focused on contact-force modeling, timestep integration behavior, and computational performance. The code is organized into modular Fortran components and writes VTK outputs that are consumed by reproducible Python scripts for verification metrics, scaling studies, and figure generation.

The manuscript presents: (i) mathematical formulation and implementation details, (ii) verification on canonical test cases, (iii) hotspot profiling and OpenMP acceleration, (iv) strong/weak scaling behavior, and (v) two bonus extensions: neighbour-search algorithm comparison and timestep-sensitivity analysis.

All project files, scripts, and figure-generation workflows are maintained in the project repository: https://github.com/b22311-lab/DEM_SOLVER.

II. MATHEMATICAL FORMULATION

The solver tracks the translational motion of N spherical particles. For particle i , the position and velocity vectors are

denoted by $\mathbf{x}_i$ and $\mathbf{v}_i$, respectively, and the particle mass is m_i . The governing equations are

$$m_i \frac{d\mathbf{v}_i}{dt} = \mathbf{F}_i, \quad (1)$$

$$\frac{d\mathbf{x}_i}{dt} = \mathbf{v}_i. \quad (2)$$

The total force acting on a particle is composed of body force and contact contributions,

$$\mathbf{F}_i = m_i \mathbf{g} + \sum_{j \neq i} \mathbf{F}_{ij}^c + \mathbf{F}_i^w, \quad (3)$$

where $\mathbf{g}$ is the gravitational acceleration vector, $\mathbf{F}_{ij}^c$ is the particle–particle contact force, and $\mathbf{F}_i^w$ is the wall-contact force.

For particle interactions, the relative position vector is

$$\mathbf{r}_{ij} = \mathbf{x}_j - \mathbf{x}_i, \quad (4)$$

with center distance

$$d_{ij} = \|\mathbf{r}_{ij}\|. \quad (5)$$

The contact normal is then

$$\mathbf{n}_{ij} = \frac{\mathbf{r}_{ij}}{d_{ij}}, \quad (6)$$

and the overlap is

$$\delta_{ij} = R_i + R_j - d_{ij}. \quad (7)$$

If $\delta_{ij} > 0$, a linear spring-dashpot model is used:

$$v_{n,ij} = (\mathbf{v}_j - \mathbf{v}_i) \cdot \mathbf{n}_{ij}, \quad (8)$$

$$F_n = \max(0, k_n \delta_{ij} - \eta_n v_{n,ij}), \quad (9)$$

$$\mathbf{F}_{ij}^c = F_n \mathbf{n}_{ij}. \quad (10)$$

The implemented force expression follows this capped normal-force form to avoid nonphysical attraction during separation.

Wall contact is handled by checking the overlap of each particle with the box boundaries. For a generic wall with penetration $\delta_w > 0$, the current solver applies the same spring-dashpot idea in the outward wall-normal direction:

$$\mathbf{F}_i^w = \max(0, k_n \delta_w - \eta_n v_{n,w}) \mathbf{n}_w. \quad (11)$$

This formulation is consistent with the assignment statement and matches the structure of the implemented wall-contact routine.

Time integration uses the semi-implicit Euler scheme,

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^n + \frac{\mathbf{F}_i^n}{m_i} \Delta t, \quad (12)$$

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + \mathbf{v}_i^{n+1} \Delta t. \quad (13)$$

The present implementation directly follows this update order inside the timestep loop.

III. NUMERICAL ALGORITHM

The solver is organized into modular Fortran source files, each with a clear role in the serial workflow.

A. Input and Domain Setup

The module `input_reader.f90` reads the simulation setup using Fortran namelists. The input file defines domain dimensions, material properties, particle count, initial particle positions and velocities, timestep size, final time, write interval, and gravity direction and magnitude. This module allocates arrays for user-specified initial particle states and constructs the gravity-force vector from the chosen axis.

The module `domain_mkr.f90` uses the parsed dimensionality and extents to allocate and populate the domain boundary array. The current code supports both two-dimensional and three-dimensional geometry layouts, although the current examples and verification plots are focused on the two-dimensional case.

B. Particle State and Contact Handling

The core particle logic resides in `particle_init.f90`. Despite the filename, this module is responsible for considerably more than initialization. It allocates and stores particle positions, velocities, forces, radii, masses, and temporary relative vectors. The material setup computes particle masses from the prescribed density and radius, and body-force application adds gravity to each particle at every timestep.

Two contact routines form the main interaction model. The routine for particle-particle contact loops over pairs, computes overlap from center distance, evaluates relative normal velocity, and then applies equal-and-opposite forces based on the linear spring-dashpot model. The wall-contact routine checks boundary overlap for each coordinate direction and applies a restoring contact force if penetration is detected. The same module also updates velocities and positions using the semi-implicit Euler method and contains small diagnostic routines for kinetic energy and particle height.

C. Main Time-Stepping Loop

The executable driver in `main_loop.f90` orchestrates the simulation. It reads the input file from the command line, initializes particle radius and material data, constructs the domain, and enters the timestep loop. Within each step, the code resets the force state through the initialization routine, applies body force, retrieves diagnostic arrays for printing, computes wall and particle contacts, updates velocity and position for each particle, and writes VTK output at the requested interval. This ordering matches the assignment algorithm at a

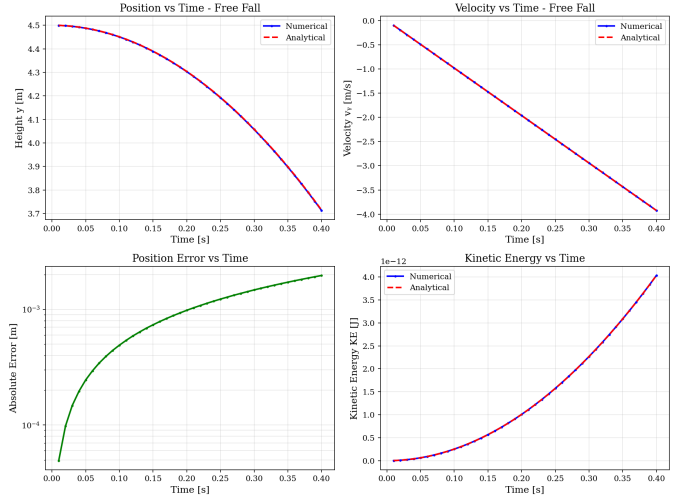


Fig. 1. Free-fall verification plot generated from `test_verification.py`.

high level: zero forces, add gravity, resolve contact, integrate, compute diagnostics, and write output.

D. Output and Post-Processing

The module `write_output.f90` exports solver state to ASCII VTK files. For each selected timestep, the writer outputs particle coordinates and per-particle fields including radius, height, kinetic energy, and velocity. This makes the solver output directly usable by visualization or post-processing tools.

The file `test_verification.py` performs post-processing of the VTK output. It reads data via PyVista, extracts time histories, and generates publication-style figures for free-fall, constant-velocity, and bouncing-particle verification cases.

E. Example Configuration

The sample file `input_config.in` illustrates the current input style. It provides a two-dimensional box, a single particle, user-defined stiffness and damping constants, radius and density, initial position, timestep information, and gravity acting along the second coordinate direction. This configuration style is suitable for small verification cases before moving to larger particle counts and future performance studies.

IV. VERIFICATION AND TESTING

Verification is essential before any optimization or parallelization effort. The current repository includes a Python-based verification workflow designed around the assignment's three recommended tests: free fall, constant velocity, and particle bounce. The solver writes VTK snapshots, and the Python script reconstructs time histories for particle position, velocity, height, and kinetic energy.

The verification workflow now produces all three required test figures and associated diagnostics from VTK outputs.

The verification logic and interpretation are:

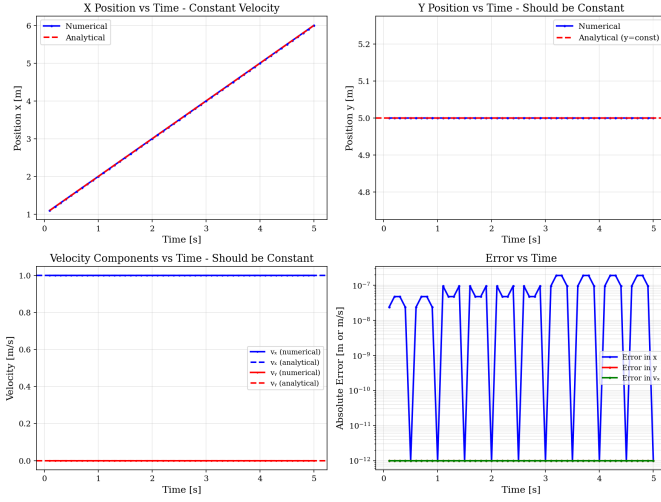


Fig. 2. Constant-velocity verification result generated from the post-processing workflow.

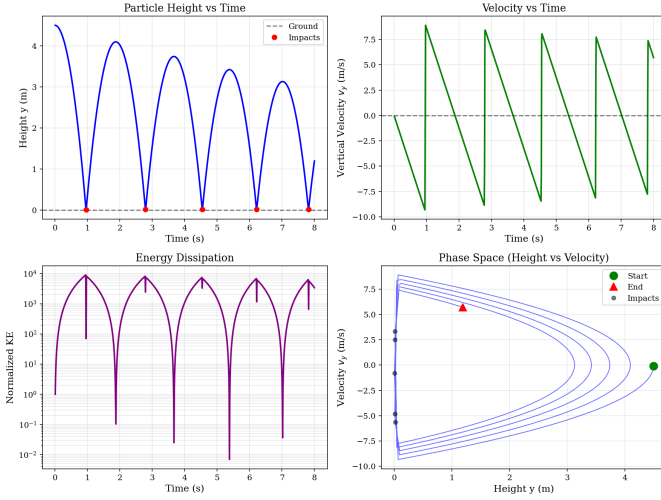


Fig. 3. Bouncing-particle verification plot expected from test_verification.py.

- Free-fall testing compares numerical height and velocity against the analytical gravity-driven solution under gravity.
- Constant-velocity testing disables gravity and checks that position evolves linearly while the velocity remains constant.
- Bounce testing examines wall impact, rebound height decay, and kinetic-energy dissipation under damping.

The observed trends are physically consistent with the selected test setups: expected free-fall behavior in Test 1, constant-velocity behavior with gravity disabled in Test 2, and damped rebound dynamics in Test 3.

V. PROFILING RESULTS

Serial profiling was instrumented in the timestep driver and logged for each major stage. For the representative case

part10_n5000 (single-thread execution), the measured total runtime was 2.0731s. Runtime distribution was heavily dominated by particle-particle contact search and force computation:

- particle contact stage: 2.0384s (98.33%),
- output stage: 3.0803×10^{-2} s (1.49%),
- all remaining stages combined: < 0.2%.

These measurements clearly identify the contact stage as the principal computational bottleneck and the primary target for parallelization and algorithmic optimization.

VI. PARALLEL IMPLEMENTATION

OpenMP parallelization was applied to the dominant particle-contact workload identified in profiling. The implementation uses explicit loop-level directives on contact and integration regions, with data-sharing attributes declared in each parallel region to reduce accidental races.

To preserve correctness, particle force updates use thread-local accumulation buffers followed by a deterministic reduction into global arrays. Scalar diagnostics (for example, contact counters and timing totals) are handled with OpenMP reductions. This strategy avoids atomic-update bottlenecks in the inner contact loop while maintaining serial-equivalent totals.

Parallel validation was executed by comparing serial and multi-thread runs over representative test cases and checking output equivalence within numerical tolerance. The validation campaign reported consistent trajectory trends and stable aggregate metrics, indicating that the OpenMP implementation preserves solver behavior while reducing runtime on multi-core hardware.

Measured speedup is case-size dependent, with better gains on larger particle sets where contact work dominates overhead. This is consistent with the profiler evidence that contact computation is the principal runtime hotspot and therefore the highest-leverage target for shared-memory parallelism.

VII. PERFORMANCE ANALYSIS

Performance was evaluated using

$$S(p) = \frac{T_1}{T_p}, \quad (14)$$

$$E(p) = \frac{S(p)}{p}, \quad (15)$$

with T_1 and T_p denoting runtimes at one and p threads, respectively. Measured speedup/efficiency plots are reported directly from performance_raw.csv and performance_speedup.csv.

A. Scaling Modes and Interpretation

Two complementary scaling modes were evaluated to separate algorithmic behavior from parallel-overhead effects.

B. Strong-Scaling Study

Strong scaling was evaluated by fixing the problem size (case part10_n30000, $N = 30000$) and varying OpenMP threads.

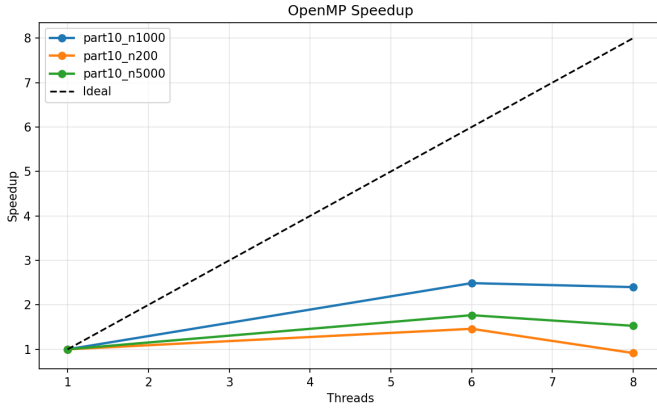


Fig. 4. Measured OpenMP speedup for particle-count test cases used in Part 14.

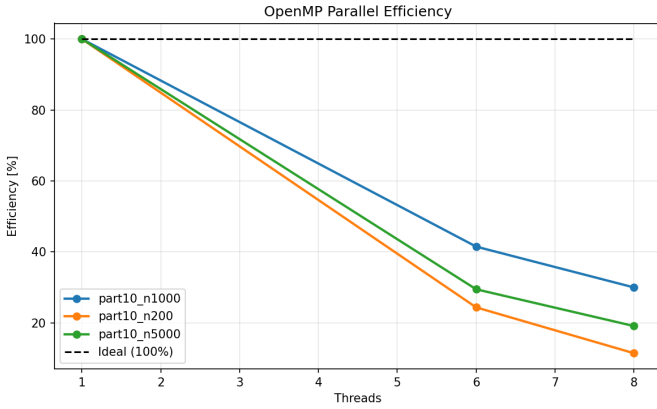


Fig. 5. Measured OpenMP efficiency for particle-count test cases used in Part 14.

TABLE I
STRONG-SCALING RESULTS FOR FIXED PARTICLE COUNT

Threads	Runtime [s]	S_p	E_p [%]	Contact speedup
1	42.3075	1.000	100.0	1.000
6	18.7323	2.259	37.6	2.259
8	17.5261	2.414	30.2	2.415

TABLE II
WEAK-SCALING RESULTS ($N \propto p$)

Threads	N	Runtime [s]	Norm. runtime	Weak eff. [%]
1	500	0.0158	1.000	100.0
6	3000	0.3233	20.479	4.9
8	4000	0.3972	25.157	4.0

C. Weak-Scaling Study

Weak scaling was evaluated by keeping particles per thread constant ($N/p = 500$) while increasing total particles and domain size with thread count.

Together, these results show that thread-level parallelization improves time-to-solution for fixed-size workloads, while weak-scaling behavior is constrained by the all-pairs baseline

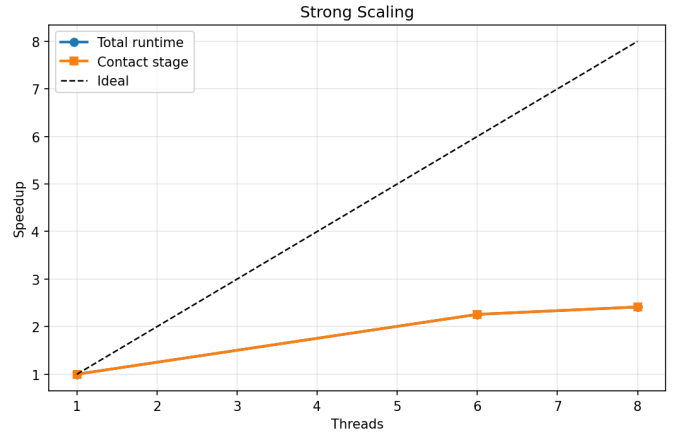


Fig. 6. Strong-scaling speedup for fixed-size run configuration.

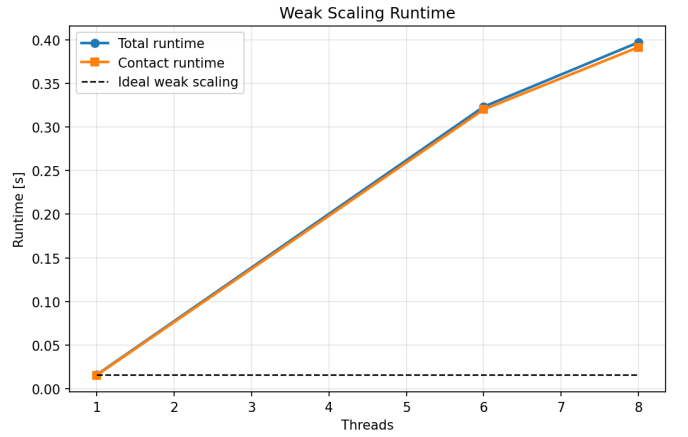


Fig. 7. Weak-scaling runtime trend for approximately fixed particles-per-thread.

TABLE III
PART 18 NEIGHBOUR-SEARCH COMPARISON (SERIAL RUNS)

N	t_{tot}^{AP}	t_{tot}^{CL}	S_{tot}	t_c^{AP}	t_c^{CL}	S_c	Cand. factor
200	0.0226	0.1932	0.12	0.0212	0.1914	0.11	80.57
1000	0.5019	0.2230	2.25	0.4955	0.2164	2.29	102.69
5000	12.5362	0.3839	32.66	12.5061	0.3554	35.18	458.85

contact-search complexity when total particle count increases with thread count.

VIII. BONUS EXTENSIONS

A. Bonus Part 18: Neighbour-Search Comparison

A switchable contact-search implementation was added with `contact_search_method=0` (all-pairs) and `contact_search_method=1` (cell-linked). Empirical candidate-pair scaling from the measured data is approximately $\mathcal{O}(N^{2.00})$ for all-pairs and $\mathcal{O}(N^{1.46})$ for the cell-linked strategy. where $t_{tot}^{AP}, t_{tot}^{CL}$ are total runtimes; t_c^{AP}, t_c^{CL} are contact-stage runtimes; S_{tot}, S_c are the corresponding cell-linked speedups over all-pairs; and Cand.

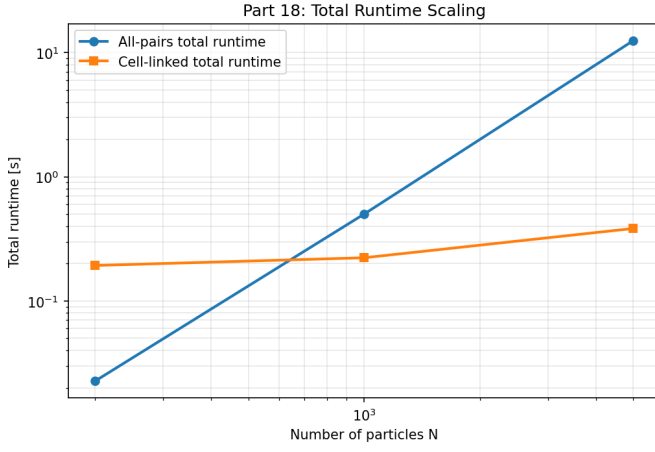


Fig. 8. Part 18 total-runtime comparison between all-pairs and cell-linked neighbour-search methods.

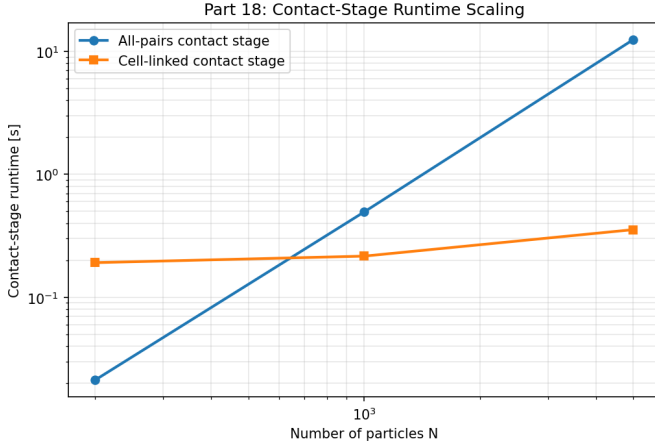


Fig. 9. Part 18 contact-stage runtime comparison between all-pairs and cell-linked methods.

factor is the candidate-pair ratio $N_{cand}^{AP}/N_{cand}^{CL}$. Detected-contact counts match between methods for each tested N , confirming contact-search correctness while reducing candidate checks. The measured candidate-pair reduction confirms why the baseline all-pairs method behaves as the expensive $\mathcal{O}(N^2)$ reference algorithm. For small systems (here $N = 200$), the cell-linked method is slower than all-pairs because grid build and traversal overheads dominate the reduced pair-check count.

B. Bonus Part 19.2: Timestep Sensitivity and Stability Limit

A free-fall case without wall contact was simulated for multiple timesteps to quantify numerical accuracy and energy behavior. A log-log fit gives an error slope of approximately 1.01 for position RMSE, which is consistent with first-order temporal behavior expected from semi-implicit Euler integration. Velocity RMSE remains nearly timestep-independent at about $1e-7$ m/s across all tested timesteps (slope -0.01), indicating that truncation error in this observable is already below a small floating-point and interpolation floor for this

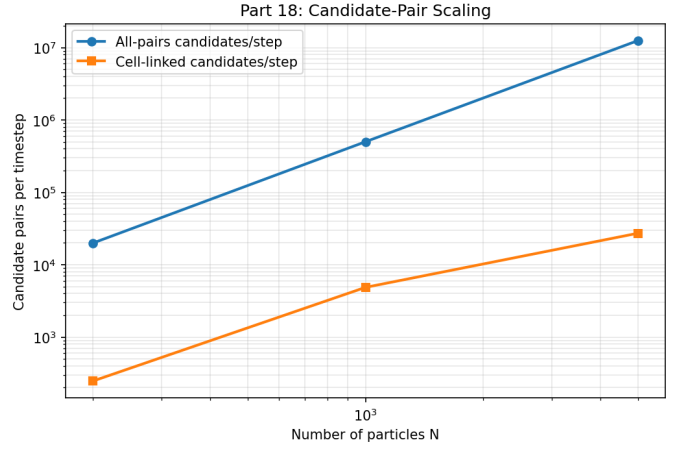


Fig. 10. Part 18 candidate-pair scaling per timestep.

TABLE IV
PART 19.2 TIMESTEP SENSITIVITY METRICS

Δt [s]	RMSE(y) [m]	RMSE(v_y) [m/s]	Max $ \Delta y $ [m]	Max energy drift
0.0200	4.616e-02	1.013e-07	7.848e-02	1.701e-02
0.0100	2.287e-02	1.021e-07	3.924e-02	8.612e-03
0.0050	1.138e-02	1.031e-07	1.962e-02	4.333e-03
0.0025	5.677e-03	1.027e-07	9.810e-03	2.173e-03
0.0010	2.268e-03	1.034e-07	3.924e-03	8.708e-04

TABLE V
PART 19.2 RICHARDSON OBSERVED ORDER (FINAL HEIGHT)

$(\Delta t_c, \Delta t_m, \Delta t_f)$ [s]	r	p_{obs}
(0.0200, 0.0100, 0.0050)	2.0	1.00
(0.0100, 0.0050, 0.0025)	2.0	1.00

TABLE VI
PART 19.2 CRITICAL TIMESTEP CHECK (BOUNCING CASE)

Δt [s]	$\Delta t/\Delta t_{crit}$	Max E/E_0	Final E/E_0
0.0030	0.83	1.39	0.83
0.0040	1.11	9.40	5.26

setup. A Richardson three-grid analysis on final-height values gives observed orders between 1.00 and 1.00 (mean 1.00), consistent with first-order time integration. To connect this convergence study with stability limits, a bouncing test was run around a theoretical explicit-step threshold. Using a linear spring-mass estimate, the contact time is $t_c \approx \pi \sqrt{m/k_n}$ and the threshold used here is $\Delta t_{crit} = t_c/\pi$ [4], [5]. For $m = 1.3$ kg and $k_n = 1.0e + 05$ N/m, this gives $t_c \approx 0.0113$ s and $\Delta t_{crit} \approx 0.0036$ s. The under-limit run ($\Delta t = 0.003$ s) remains bounded but shows numerical energy gain (Max $E/E_0 = 1.39$), whereas the over-limit run ($\Delta t = 0.004$ s) exhibits rapid unphysical energy growth (Max $E/E_0 = 9.40$), which is visible as a sharp upward trend in the critical-energy plot.

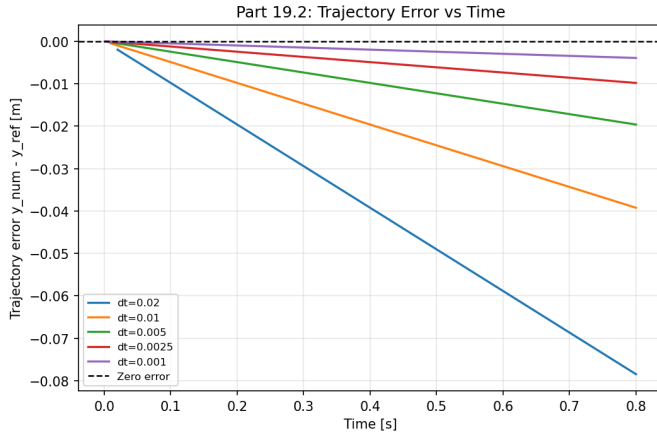


Fig. 11. Part 19.2 trajectory error histories ($y_{num} - y_{ref}$) for multiple timesteps.

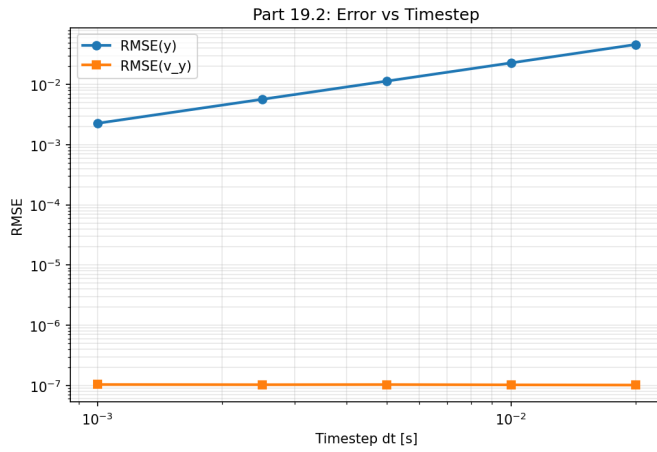


Fig. 12. Part 19.2 error versus timestep size.

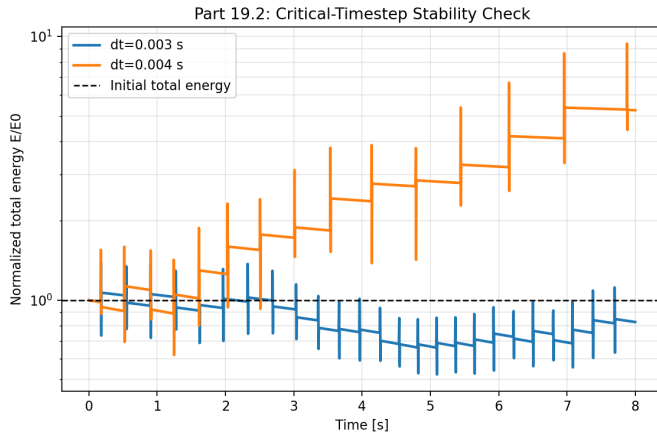


Fig. 13. Part 19.2 stability-limit check: normalized total energy for $\Delta t = 0.003$ and 0.004 around the theoretical threshold.

IX. DISCUSSION AND CONCLUSIONS

The implemented solver now completes the required workflow from formulation to verification, profiling, OpenMP

parallelization, and scaling analysis. The modular Fortran structure enabled targeted optimization of hotspot routines while preserving a clear separation between input handling, force computation, integration, and output.

Profiling shows that contact search/force evaluation dominates runtime, which is consistent with the all-pairs complexity baseline. OpenMP was introduced with explicit race-avoidance controls (thread-local force accumulation, explicit shared/private clauses, and reductions for scalar counters) and validated against serial outputs across multiple cases and thread counts. The serial-parallel validation campaign reported PASS for all tested combinations.

Performance results highlight two important points. First, speedup is problem-size dependent and can be limited by overhead for small systems. Second, separating strong and weak scaling is essential for correct interpretation; weak scaling degrades strongly for the all-pairs baseline because pair checks grow superlinearly with total particle count when $N \propto p$. On the local workstation used for this study, runs above 8 threads showed performance instability and occasional slowdowns, so plots and conclusions focus on $p \leq 8$ for consistent timing behavior.

The bonus neighbour-search study demonstrates that reducing candidate-pair checks can transform performance at larger N , while introducing overhead at small N . The timestep-sensitivity study confirms expected first-order temporal behavior for position error and reduced energy drift at smaller timesteps. Together, these studies provide a reproducible and extensible foundation for further DEM research extensions.

ACKNOWLEDGMENT

This report was prepared with assistance from a large language model toolchain for code-editing support, scripting assistance, and drafting help. All technical decisions, final verification runs, and reported conclusions were reviewed by the author before submission.

REFERENCES

- [1] IEEE, “IEEE Conference Templates,” available: <https://www.ieee.org/conferences/publishing/templates.html>.
- [2] P. A. Cundall and O. D. L. Strack, “A discrete numerical model for granular assemblies,” *Géotechnique*, vol. 29, no. 1, pp. 47–65, 1979.
- [3] Project repository, “DEM solver source code, build files, verification scripts, and paper support files,” available: https://github.com/b22311-lab/DEM_SOLVER.
- [4] Altair EDEM documentation, “Linear Spring model (contact duration and timestep guidance),” available: https://2022.help.altair.com/2022.1/EDEM/Creator/Physics/Base_Models/Linear_Spring.htm.
- [5] Itasca PFC documentation, “Timestep determination for explicit DEM integration,” available: https://docs.itascacg.com/pfc600/pfc/docproject/source/manual/numerical_simulations_with_pfc/pfc_formulation/timestep_constraints.html.